Configuration Management Plan

AuralAid: Adaptive AI Hearing with Contextual Understanding

Version 1.0

Prepared for	Final Year Project Documentation
Project	AuralAid Smart Hearing and Contextual Assistance System
Prepared by	Ayesha Nazir (22-SE-079) Hamza Moeen (22-SE-025)
Supervisor	Mr. Junaid Ali Khan
Discipline	BS Software Engineering, HITEC University Taxila

Department of Computer Science
HITEC University Taxil

Table of Contents

Table of Contents ..... **Error! Bookmark not defined.**

1. Scope and References 3

1.1 Scope 3

1.2 References 3

2. Definitions and Acronyms 3

2.1 Definitions 3

2.2 Acronyms 4

3. Software Configuration Management Plan 5

3.1 Introduction 5

3.1.1 Purpose 5

3.1.2 Scope 5

3.1.3 Definitions and Acronyms 5

3.1.4 References 5

3.2 Management 5

3.2.1 Organization 5

3.2.2 SCM Responsibilities 6

3.2.3 Interface Control 6

3.2.4 SCMP Implementation 7

3.2.5 Applicable Policies, Directives, and Procedures 7

3.3 SCM Activities 7

3.3.1 Configuration Identification 7

3.3.2 Configuration Control 8

3.3.3 Configuration Status Accounting 9

3.3.4 Audits and Reviews 10

3.4 Tools, Techniques and Methodologies 10

3.5 Supplier Control 11

3.6 Records Collection and Retention 11

1. Scope and References

1.1 Scope

The scope of this Configuration Management Plan (CMP) is to define how all software, firmware, AI, mobile application, data, and documentation assets for the AuralAid project will be identified, controlled, versioned, verified, and released throughout the full project life cycle. AuralAid is not a conventional web-only application; it is a multi-component assistive system that combines an ESP32-S3 embedded platform, audio signal processing, Bluetooth/Wi-Fi connectivity, AI-assisted speech and emotion analysis, health sensor integration, a Flutter mobile application, and optional cloud-backed synchronization. The configuration approach must therefore support both software engineering discipline and embedded-system traceability.

- Lifecycle coverage: planning, requirements, design, implementation, integration, verification, pilot demonstration, maintenance, and future enhancement.
- Configuration items covered: embedded firmware, DSP modules, AI model files and training artifacts, Flutter mobile application source code, backend or cloud connectors, APIs, BLE/Wi-Fi configuration profiles, hardware pin maps, interface control documents, test scripts, datasets, user manuals, and release notes.
- Change control coverage: functional changes, bug fixes, model updates, calibration profile updates, mobile UI changes, database or cloud schema changes, and security or privacy-impacting updates.
- Status and traceability coverage: every approved baseline, version, defect, change request, test result, and release candidate will be logged so the team can always identify what was built, what was tested, and what was deployed.
- Risk-sensitive scope: because the system includes health indicators, live captions, environmental noise alerts, and assistive communication features, configuration control must protect user safety, data integrity, and predictable system behavior.

1.2 References

- IEEE 828 – Configuration Management in Systems and Software Engineering.
- ISO/IEC/IEEE 12207 – Systems and software life cycle processes.
- IEEE 29148 – Requirements engineering practices for system and software projects.
- Project Proposal: AuralAid – Adaptive AI Hearing with Contextual Understanding.
- Repository standards, branch policies, coding guidelines, issue logs, and release procedures defined for the AuralAid project.

2. Definitions and Acronyms

2.1 Definitions

Baseline: A formally reviewed and approved configuration snapshot that serves as the reference point for further work and can only be changed through controlled procedures.

Configuration Item (CI): Any software, firmware, document, dataset, model, script, hardware definition, or environment setting placed under formal configuration management.

Configuration Control Board (CCB): The decision-making body that reviews, approves, defers, or rejects significant project changes.

Digital Signal Processing (DSP) Module: The firmware or software component responsible for audio filtering, noise reduction, speech enhancement, and sound-level analysis.

Interface Control Document (ICD): A controlled document describing the structure, protocol, and behavior of interfaces between hardware, firmware, app, cloud, and external services.

Release Candidate: A build or package that has passed planned checks and is ready for final acceptance testing or demonstration.

Traceability: The ability to relate requirements, design decisions, source code, test cases, defects, and releases to one another.

Model Artifact: A trained or converted AI file, such as speech-to-text, emotion-detection, or summarization model assets and their version metadata.

Controlled Library: The approved repository or storage location for baselined project items.

2.2 Acronyms

Acronym	Meaning
AI	Artificial Intelligence
ASR	Automatic Speech Recognition
BLE	Bluetooth Low Energy
CCB	Configuration Control Board
CI	Configuration Item
CMP	Configuration Management Plan
DSP	Digital Signal Processing
ICD	Interface Control Document
PII	Personally Identifiable Information
QA	Quality Assurance
SCM	Software Configuration Management
SCMP	Software Configuration Management Plan
SER	Speech Emotion Recognition
STT	Speech-to-Text
UAT	User Acceptance Testing
WER	Word Error Rate

3. Software Configuration Management Plan

3.1 Introduction

AuralAid requires disciplined configuration management because it is a cross-domain assistive platform rather than a single software package. The project includes embedded firmware on ESP32-S3, AI-assisted processing on mobile or companion compute resources, a Flutter-based application interface, sensor integration for heart rate, SpO_2, and temperature, and safety-driven alerting. This SCMP defines the controls needed to preserve consistency between all of these parts.

3.1.1 Purpose

The purpose of this plan is to ensure that all AuralAid changes are proposed, evaluated, implemented, verified, documented, and released in a controlled manner. The plan protects quality, regression stability, privacy, and traceability while enabling the team to iterate on firmware, AI models, and user-facing features without losing configuration integrity.

3.1.2 Scope

- Embedded firmware: audio capture, DSP, sensor acquisition, alert logic, BLE/Wi-Fi communication, configuration parameters, and hardware abstraction layers.
- Mobile application: Flutter UI, connectivity logic, caption display, emotion indicators, health dashboards, alert views, and local storage.
- AI and data assets: speech-to-text models, emotion-detection models, summarization assets if used, feature-extraction pipelines, datasets, evaluation reports, and converted deployment formats.
- Infrastructure and integration: API keys, cloud connectors, data schemas, route-finder integrations, release packaging scripts, and deployment manifests.
- Documentation: project proposal, SRS, architecture diagrams, DFDs, use cases, test plans, user instructions, calibration procedures, and release records.

3.1.3 Definitions and Acronyms

All definitions and acronyms used in this SCMP are provided in Section 2 and apply uniformly across software, firmware, AI, and documentation items.

3.1.4 References

- AuralAid project proposal and associated architecture, requirements, and milestone descriptions.
- Project-level branch naming standards, semantic versioning rules, issue-tracking conventions, and review policies.

3.2 Management

3.2.1 Organization

The AuralAid project will use a lightweight but formal configuration structure suitable for a final year prototype with multi-disciplinary components.

Role / Unit	Primary Focus	Configuration Relevance
Project Team	Requirements, implementation, integration, and final delivery	Creates and updates controlled items; submits change requests and release candidates.

SCM Lead / Configuration Owner	Version control discipline, baselines, release records, and change logs	Maintains repositories, tags, baseline registers, and configuration status reports.
QA and Verification Function	Functional, integration, performance, and acceptance testing	Confirms that each build matches approved requirements and test evidence.
Supervisor / Review Authority	Academic guidance and milestone oversight	Reviews major baseline approvals, milestone readiness, and critical scope changes.
User Evaluation Stakeholders	Prototype feedback and usability validation	Participate in UAT, caption clarity checks, and alert usefulness validation.

3.2.2 SCM Responsibilities

- Identify all configuration items and maintain a configuration index for firmware, app, AI, datasets, documents, and release packages.
- Define and protect baselines for requirements, design, implementation, integration, test, and release.
- Administer branch protection, review requirements, tagging, and release packaging.
- Ensure that critical changes affecting health alerts, speech processing, privacy, or external APIs receive formal review before merge or deployment.
- Maintain the mapping between requirements, code branches, issues, test cases, defects, and released versions.
- Archive approved build outputs, model versions, configuration files, and demonstration packages so that each milestone can be reproduced.

Major change authority will remain with a small Configuration Control Board consisting of the project team, QA representative, and supervisor for scope-critical decisions.

3.2.3 Interface Control

- Hardware-to-firmware interfaces: microphone inputs, sensor buses, motor or haptic outputs, battery and power states, and ESP32-S3 pin configuration.
- Firmware-to-mobile interfaces: BLE services, message formats, payload sizes, reconnection behavior, error codes, and timing expectations.
- Mobile-to-cloud or external API interfaces: authentication, request/response structures, route-finder data, optional transcript synchronization, and privacy controls.
- AI pipeline interfaces: feature formats, model input/output tensor shape or schema, quantized model versions, and performance thresholds.

Any interface change that breaks compatibility, changes payload structures, modifies sensor mappings, or affects safety behavior shall require an ICD update, regression testing, and approval before release.

3.2.4 SCMP Implementation

Milestone	Configuration Output	Approval Gate	Evidence Required
Requirements baseline	Proposal, scope, business objectives, initial requirements set	Supervisor + project team	Approved requirements set and issue register
Design baseline	Architecture, use cases, ICDs, hardware/software partitioning	Project team + QA review	Design review record and updated diagrams
Implementation baseline	app, AI branches aligned to sprint or milestone	SCM lead	Tagged repository state and build verification
Integration baseline	Integrated device/app/AI build	CCB	Integration test report and defect status
Release baseline	Demonstration-ready package and documents	Supervisor + QA sign-off	Release notes, acceptance checklist, archived binaries

3.2.5 Applicable Policies, Directives, and Procedures

- Versioning policy: semantic versioning for app and software services; milestone-based tagged versions for firmware and AI model bundles.
- Repository policy: all controlled items must reside in version-controlled repositories or approved document stores with history enabled.
- Review policy: no direct modification of baselined branches; pull request review required for source changes, configuration changes, and model updates.
- Traceability policy: each change must reference a requirement, defect, enhancement ticket, or risk mitigation action.
- Security and privacy policy: credentials, tokens, and sensitive logs must never be stored in plain text within public or shared repositories.
- Quality gate policy: a release candidate cannot be baselined without required tests, unresolved critical defect review, and updated release notes.

3.3 SCM Activities

AuralAid configuration management is organized around configuration identification, control, status accounting, and audits/reviews.

3.3.1 Configuration Identification

Baseline	Life-Cycle Phase	Items Forming Baseline	Review Event	Acceptance Criteria	Participation
Functional Baseline	Requirements Definition	Proposal, objectives, scope, business needs, high-level	Requirements review	Requirements are complete, testable, and aligned to	Project team and supervisor

		architecture, initial risk list		assistive use cases	
Allocated Baseline	Design	design, app flows, ICDs, sensor mapping, AI pipeline definition, data/privacy decisions	Design review	Interfaces are defined and technically feasible	Project team with QA review
Product Baseline	Implementation and Test	Source code, model files, binaries, app packages, test scripts, user docs	Integration and release review	Features meet quality gates and evidence is complete	QA, SCM lead, supervisor

Configuration items will be labeled using a predictable identifier structure, such as FW- for modules, APP- for mobile components, AI- for models, DOC- for documents, TEST- for scripts, and REL- for release packages. Each item will include version, owner, storage location, and status.

3.3.2 Configuration Control

Life-Cycle Phase	Typical Change Authority	Examples of Controlled Changes	Approval Mode
Requirements	Project team + supervisor	Scope change, feature addition, health-monitoring expansion, safe-route scope change	Formal review
Design	Design owner + QA review	BLE payload update, sensor architecture change, ICD revision, storage design revision	Recorded design decision
Implementation	Module owner + reviewer	Firmware bug fix, app UI update, AI inference optimization, config tuning	Pull request review
Integration/Test	SCM lead + QA	Build candidate promotion, model package replacement, calibration profile update	Test-gated approval
Release/Maintenance	CCB	Critical bug fix, privacy patch, field parameter change, post-demo stabilization	Release approval

- All change proposals shall include change description, reason, affected items, impact assessment, test needs, rollback considerations, and decision status.
- Changes affecting speech recognition accuracy, noise alert thresholds, health alert thresholds, BLE connectivity, privacy handling, or safety-related notifications will be treated as high-impact changes.
- Emergency fixes may be fast-tracked, but they must still be documented, tested, and retrospectively reviewed before being merged into the release baseline.
- Third-party library or API changes require compatibility assessment, license review, and regression verification.

3.3.3 Configuration Status Accounting

Configuration status accounting will provide continuous visibility into what exists, what changed, what has been tested, and what is ready for release.

- Records maintained: CI register, change request log, branch/tag history, issue tracker, build logs, test execution results, defect register, release notes, and audit findings.
- Report cadence: weekly status summaries during active development; milestone reports for design, integration, and release gates.
- Traceability matrix coverage: requirement-to-module, requirement-to-test, defect-to-fix, and release-to-baseline mappings.
- Dashboard indicators: open defects by severity, pending change requests, model version in use, release candidate status, test pass rate, and unresolved interface issues.

Project-specific quality metrics and configuration status measures

Metric	Target / Threshold	Configuration Relevance	Evidence Source
Speech-to-text accuracy	≥ 90% caption accuracy in planned test scenarios	Determines acceptance of ASR model or service baseline	Test dataset report / UAT results
Emotion detection accuracy	≥ 80% on defined evaluation set	Controls acceptance of SER model version	Model evaluation report
Noise alert response time	< 2 seconds above threshold event	Validates safety-related firmware/app behavior	Latency test logs
Speech clarity / enhancement	Target aligned with proposal goal; measurable improvement in noisy conditions	Approves DSP tuning changes	Controlled audio test report
Build reproducibility	100% reproducible tagged release build	Ensures released binaries map to source baseline	CI/build logs
Critical defect leakage	0 open critical defects in release baseline	Release gate protection	Defect tracker

BLE/Wi-Fi connection stability	Successful reconnect and stable transfer in planned scenarios	Protects interface baseline	Connectivity regression report
Health sensor data validity	Stable readings within accepted calibration range	Controls sensor configuration approval	Calibration and verification logs

3.3.4 Audits and Reviews

- Functional configuration audit: confirms that the implemented build satisfies approved requirements for captioning, emotion indication, noise alerts, health monitoring, and connectivity.
- Physical configuration audit: confirms that the exact released firmware, app package, model artifact, configuration file, and documentation set match the recorded baseline.
- Interface review: verifies BLE services, API schemas, ICDs, and sensor mappings after interface-impacting changes.
- Security and privacy review: checks secret handling, log content, permission usage, and cloud data controls where applicable.
- Demonstration readiness review: ensures the project team can reproduce the approved build and supporting documentation for evaluation or presentation.

Audit findings shall be logged with owner, due date, severity, and closure evidence. No release baseline shall be closed while critical audit actions remain unresolved unless explicitly approved with rationale.

3.4 Tools, Techniques and Methodologies

Category	Tools / Techniques	SCM Usage
Version Control	Git / GitHub	Branching, pull requests, tags, release records, issue linkage
Embedded Development	ESP-IDF / Arduino IDE, FreeRTOS, ESP-DSP	Firmware source control, build management, parameter versioning
Mobile Development	Flutter SDK	Controlled app builds, release candidate packaging, UI regression tracking
AI / ML	TensorFlow Lite, PyTorch, conversion scripts	Model artifact control, evaluation traceability, deployment packaging
Testing	Unit tests, integration tests, hardware-in-the-loop tests, UAT scripts	Evidence for change approval and baseline acceptance
Documentation	Architecture diagrams, ICDs, SRS, release notes	Controlled document baselines and review history
Storage / Build Control	Approved cloud or local artifact repository	Controlled binaries, APKs, firmware images, model files, datasets

Testing strategy tailored to AuralAid

- Unit testing for firmware modules, sensor drivers, BLE services, app logic, and helper utilities.
- Integration testing across microphone input, DSP processing, AI output, app display, health sensor updates, and alert generation.
- Hardware-in-the-loop testing for audio pipelines, dB alerts, haptic feedback, and communication timing under realistic sensor conditions.
- AI validation for ASR and emotion detection using curated datasets and controlled noisy-speech scenarios.
- Regression testing after any change to timing, payload format, model version, alert threshold, or UI presentation.
- User acceptance testing focused on readability of captions, clarity of alerts, wearable usability, and perceived usefulness in noisy environments.

Compliance and assurance orientation for this project

- IEEE 828 alignment for configuration planning, identification, control, status accounting, and audits.
- ISO/IEC/IEEE 12207 alignment for life-cycle process discipline across development, verification, release, and maintenance activities.
- Requirements traceability practices consistent with modern software engineering documentation and review expectations.
- Privacy-aware handling of voice, transcript, and health-related records, including minimal retention of sensitive test data and controlled access to stored logs.
- Academic prototype compliance posture: this project is managed as a research and assistive technology prototype, not as a certified medical device; therefore, any health indications are informational within prototype scope and must be labeled accordingly in documentation and demos.

3.5 Supplier Control

AuralAid uses third-party hardware modules, libraries, datasets, APIs, and possibly cloud services. These externally sourced items must be controlled so they do not introduce licensing, compatibility, privacy, or reliability problems.

- All third-party components shall be listed with version, source, license, and intended use.
- External datasets and pre-trained models shall be documented with provenance, preprocessing notes, and evaluation limitations.
- API-based features such as speech services, maps, or synchronization services shall be version-tracked and tested for fallback behavior when unavailable.
- Supplier or package updates shall be reviewed before adoption, especially where they affect permissions, inference output, latency, or cost.
- Only approved third-party assets may be included in release baselines; ad hoc dependencies are not permitted in final packaging.

3.6 Records Collection and Retention

- Records to retain: baselines, CI register, change requests, design decisions, issue logs, test evidence, model evaluation reports, release notes, audit findings, and approved deliverables.
- Storage approach: controlled digital repositories with access restriction, backup, and version history.

- Retention period for academic project records: maintained through the active project life cycle and preserved for at least the full academic evaluation cycle, with important release and audit records kept longer if required by department policy.
- Sensitive data handling: raw voice samples, transcripts, health logs, and user evaluation records used in testing must be stored minimally, access-controlled, and anonymized where practical.
- Final archive content: approved source snapshot, firmware image, app package, model bundle, key documents, acceptance evidence, and a final configuration index.